

GESTION DE FLUX EN JAVA

Dr. Ing Mohamed Mazouzi

Introduction

2

- Un programme a souvent besoin d'échanger des informations, que ce soit pour recevoir des données d'une source ou pour envoyer des données vers un destinataire.
- La source et la destination de ces échanges peuvent être de natures multiples : fichier, socket réseau, un autre programme, etc ...
- De la même façon, la nature des données échangées peut être diverse : du texte, des images, du son, etc ...

La présentation des flux

3

- Les flux (streams en anglais) permettent d'encapsuler ces processus d'envoi et de réception de données.
- Les flux traitent toujours les données de façon séquentielle.
- En Java, les flux peuvent être divisés en plusieurs catégories :
 - ▣ les flux d'entrée (input stream) et les flux de sortie (output stream)
 - ▣ les flux de traitement de caractères et les flux de traitement d'octets

La présentation des flux

4

- Java définit des flux pour lire ou écrire des données mais aussi des classes qui permettent de faire des traitements sur les données du flux.
- Ces classes doivent être associées à un flux de lecture ou d'écriture et sont considérées comme des filtres.
 - ▣ Par exemple, il existe des filtres qui permettent de mettre les données traitées dans un tampon (buffer) pour les traiter par lots.
- Toutes ces classes sont regroupées dans le package `java.io`

Les classes de gestion des flux

5

- ❑ Ce qui déroute dans l'utilisation de ces classes, c'est leur nombre et la difficulté de choisir celle qui convient le mieux en fonction des besoins.
- ❑ Pour faciliter ce choix, il faut comprendre la dénomination des classes : cela permet de sélectionner la ou les classes adaptées aux traitements à réaliser.
- ❑ Le nom des classes se compose d'un **préfixe** et d'un **suffixe**.
- ❑ Il y a quatre **suffixes** possibles en fonction du type de flux (flux d'octets ou de caractères) et du sens du flux (entrée ou sortie).

Les classes de gestion des flux

6

	Flux d'octets	Flux de caractères
Flux d'entrée	InputStream	Reader
Flux de sortie	OutputStream	Writer

- Il existe donc quatre hiérarchies de classes qui encapsulent des types de flux particuliers.
- Ces classes peuvent être séparées en deux séries de deux catégories différentes : les classes de lecture et d'écriture et les classes permettant la lecture de caractères ou d'octets.

Les classes de gestion des flux

7

- les sous-classes de **Reader** sont des types de flux en **lecture** sur des ensembles de **caractères**
- les sous-classes de **Writer** sont des types de flux en **écriture** sur des ensembles de **caractères**
- les sous-classes de **InputStream** sont des types de flux en **lecture** sur des ensembles **d'octets**
- les sous-classes de **OutputStream** sont des types de flux en **écriture** sur des ensembles **d'octets**

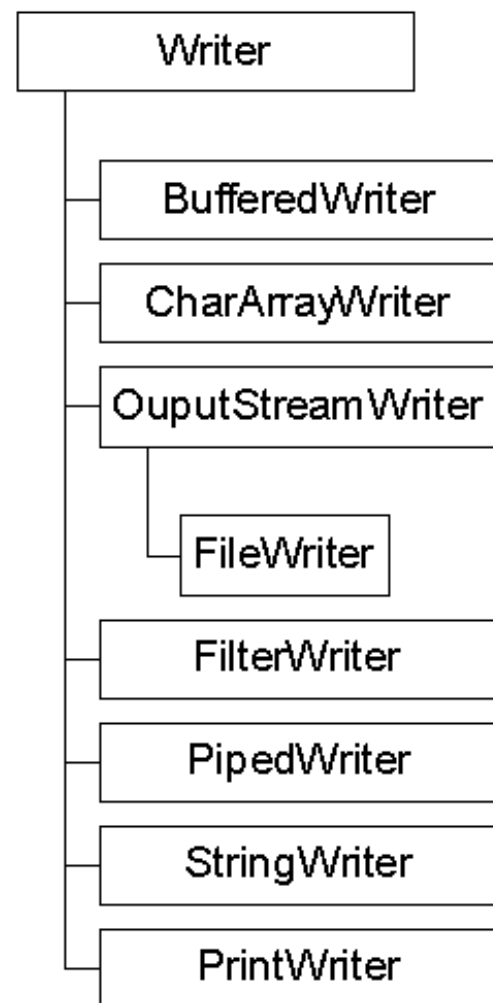
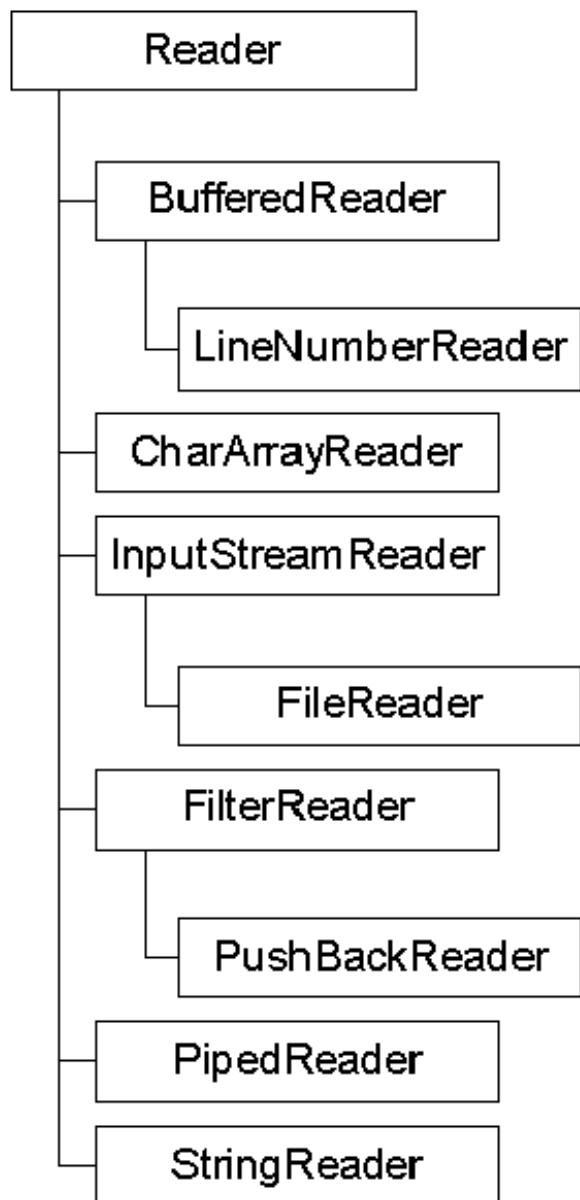
Les classes de gestion des flux

8

- Pour les *filtres*, le **préfixe** contient le type de traitement qu'il effectue. Les filtres n'existent pas obligatoirement pour des flux en entrée et en sortie. Exemple:
 - Buffered : ce type de filtre permet de mettre les données du flux dans un tampon. Il peut être utilisé en entrée et en sortie
 - Data : ce type de flux permet de traiter les octets sous forme de type de données
 - LineNumber : ce filtre permet de numérotter les lignes contenues dans le flux
 - Print : ce filtre permet de réaliser des impressions formatées
 - Object : ce filtre est utilisé par la sérialisation
 - InputStream / OutputStream : ce filtre permet de convertir des octets en caractères

LES FLUX DE CARACTÈRES





Les flux de caractères: La classe Reader

11

- C'est une classe abstraite qui est la classe mère de toutes les classes qui gèrent des flux de **caractères** en **lecture**. Cette classe définit plusieurs méthodes exemples :

Méthodes	Rôles
<code>boolean ready()</code>	indique si le flux est prêt à être lu
<code>close()</code>	ferme le flux et libère les ressources qui lui étaient associées
<code>int read()</code>	renvoie le caractère lu ou -1 si la fin du flux est atteinte.
<code>int read(char[])</code>	lire plusieurs caractères et les mettre dans un tableau de caractères
<code>long skip(long)</code>	saute autant de caractères dans le flux que la valeur fournie en paramètre. Elle renvoie le nombre de caractères sautés.

Les flux de caractères: La classe **Writer**

12

- C'est une classe abstraite qui est la classe mère de toutes les classes qui gèrent des flux de **caractères** en **écriture**. Cette classe définit plusieurs méthodes exemples :

Méthodes	Rôles
<code>close()</code>	ferme le flux et libère les ressources qui lui étaient associées
<code>write(int)</code>	écrire le caractère en paramètre dans le flux.
<code>write(char[])</code>	écrire le tableau de caractères en paramètre dans le flux.
<code>write(char[], int, int)</code>	écrire plusieurs caractères. Elle attend en paramètres : un tableau de caractères, l'indice du premier caractère et le nombre de caractères à écrire.

Les flux de caractères avec un fichier

13

- Les classes **FileReader** et **FileWriter** permettent de gérer des flux de caractères avec des fichiers.
- ▣ Les flux de caractères en lecture sur un fichier

```
FileReader fichier = new FileReader("monfichier.txt");
```

- Il existe plusieurs méthodes de la classe `FileReader` qui permettent de lire un ou plusieurs caractères dans le flux. Toutes ces méthodes sont héritées de la classe `Reader`
- Une fois les traitements sur le flux terminés, il faut libérer les ressources qui lui sont allouées en utilisant la méthode `close()`.

Les flux de caractères avec un fichier

14

▣ Les flux de caractères en écriture sur un fichier

Constructeur	Rôle
FileWriter(String)	Si le nom du fichier précisé n'existe pas alors le fichier sera créé. S'il existe et qu'il contient des données celles-ci seront écrasées
FileWriter(File)	Idem mais le fichier est précisé avec un objet de la classe File
FileWriter(String, boolean)	Le booléen permet de préciser si les données seront ajoutées au fichier (valeur true) ou écraseront les données existantes (valeur false)

```
FileWriter fichier = new FileWriter ("monfichier.dat");
```

- Une fois les traitements sur le flux terminés, il faut libérer les ressources qui lui sont allouées en utilisant la méthode `close()`.

Les flux de caractères tamponnés avec un fichier.

15

- Pour améliorer les performances des flux sur un fichier, la mise en tampon des données lues ou écrites permet de traiter un ensemble de caractères représentant une ligne plutôt que de traiter les données caractères par caractères. Le nombre d'opérations est ainsi réduit.
- Les classes ***BufferedReader*** et ***BufferedWriter*** permettent de gérer des flux de caractères tamponnés avec des fichiers.

Les flux de caractères tamponnés avec un fichier.

16

□ Les flux de caractères tamponnés en lecture avec un fichier

Constructeur	Rôle
<code>BufferedReader(Reader)</code>	le paramètre fourni doit correspondre au flux à lire.
<code>BufferedReader(Reader, int)</code>	l'entier en paramètre permet de préciser la taille du buffer. Il doit être positif sinon une exception

```
BufferedReader fichier = new BufferedReader(new FileReader("monfichier.txt"));
```

□ La classe `BufferedReader` définit une méthode supplémentaire pour la lecture :

Méthode	Rôle
<code>String readLine()</code>	lire une ligne de caractères dans le flux. Une ligne est une suite de caractères qui se termine par un retour chariot '\r' ou un saut de ligne '\n' ou les deux.

Exercice

17

- Ecrire un programme permettant d'afficher ligne par ligne un fichier lu en mode flux de caractère tamponné

Corrigé de l'exercice

18

```
import java.io.*;

public class TestBufferedReader1 {

    public static void main(String args[]) throws IOException
    {
        String ligne ;
        BufferedReader fichier = new BufferedReader(new FileReader("source.txt"));
        while ((ligne = fichier.readLine()) != null)
        {
            System.out.println(ligne);
        }
        fichier.close();
    }
}
```

Les flux de caractères tamponnés avec un fichier.

19

▣ Les flux de caractères tamponnés en écriture avec un fichier

Constructeur	Rôle
<code>BufferedWriter(Writer)</code>	le paramètre fourni doit correspondre au flux dans lequel les données sont écrites.
<code>BufferedWriter(Writer, int)</code>	l'entier en paramètre permet de préciser la taille du buffer. Il doit être positif sinon une exception

```
BufferedWriter fichier = new BufferedWriter( new FileWriter("monfichier.txt"));
```

■ La classe `BufferedWriter` possède plusieurs méthodes pour aérer le flux :

Méthode	Rôle
<code>flush()</code>	vide le tampon en écrivant les données dans le flux.
<code>newLine()</code>	écrire un séparateur de ligne dans le flux

Exercice

20

- Ecrire un programme permettant de créer un fichier en utilisant `BufferedWriter` type dans lequel vous affichez par exemple Bonjour, la date, etc.

Corrigé de l'exercice

21

```
import java.io.*;
import java.util.*;

public class TestBufferedWriter1 {

    public static void main(String args[]) {
        try {
            int nombre = 123;
            BufferedWriter fichier = new BufferedWriter(new FileWriter("print.txt"));
            fichier.write("bonjour tout le monde");
            fichier.newLine();
            fichier.write("Nous sommes le " + new Date());
            fichier.write(", le nombre magique est " + nombre);
            fichier.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

La classe PrintWriter

22

- Cette classe permet d'écrire dans un flux des données formatées.

Constructeur	Rôle
<code>PrintWriter(Writer)</code>	Le paramètre fourni précise le flux. Le tampon est automatiquement vidé.

```
PrintWriter fichier = new PrintWriter( new FileWriter("monfichier.txt"));
```

- ▣ Les méthodes `write()` sont héritées de la classe `Writer` qui définit plusieurs méthodes pour envoyer des données formatées dans le flux :
- ▣ `print( ... )`: Plusieurs surcharges de la méthode `print()` acceptent des données de différents types pour les convertir en caractères et les écrire dans le flux
- ▣ `println()`: Cette méthode permet de terminer la ligne courante dans le flux en y écrivant un saut de ligne.

LES FLUX D'OCTETS



Les flux d'octets avec un fichier.

24

▣ Les flux d'octets en lecture sur un fichier

Constructeur	Rôle
<code>FileInputStream(String)</code>	Ouvre un flux en lecture sur le fichier dont le nom est donné en paramètre
<code>FileInputStream(File)</code>	Idem mais le fichier est précisé avec un objet de type <code>File</code>

```
FileInputStream fichier = new FileInputStream("monfichier.dat");
```

- Il existe plusieurs méthodes de la classe `FileInputStream` qui permettent de lire un ou plusieurs octets dans le flux.

- Exemple:

`int read()`: Cette méthode envoie la valeur de l'octet lu ou -1 si la fin du flux est atteinte.

Les flux d'octets avec un fichier

25

▣ Les flux d'octets en écriture sur un fichier

Constructeur	Rôle
<code>FileOutputStream(String)</code>	Si le fichier précisé n'existe pas, il sera créé. Si il existe et qu'il contient des données celles-ci seront écrasées.
<code>FileOutputStream(String, boolean)</code>	Le booléen permet de préciser si les données seront ajoutées au fichier (valeur true) ou écraseront les données existantes (valeur false)

```
FileOuputStream fichier = new FileOutputStream("monfichier.dat");
```

- Il existe plusieurs méthodes de la classe `FileOutputStream` qui permettent d'écrire un ou plusieurs octets dans le flux.

Méthode	Rôle
<code>write(int)</code>	Cette méthode écrit l'octet en paramètre dans le flux
<code>write(byte[])</code>	Cette méthode écrit plusieurs octets. Elle attend en paramètre : un tableau d'octets qui contient les octets à écrire : tous les éléments du tableau sont écrits

Remarque

26

- Pour améliorer les performances des flux sur un fichier, la mise en tampon des données lues ou écrites permet de traiter un ensemble d'octets plutôt que de traiter les données octet par octet. Le nombre d'opérations est ainsi réduit.